High Availability (HA) System – Hot & Standby Notes

1. System Overview

- **Goal:** Ensure zero/minimal downtime by using **hot and standby Kubernetes clusters**.
- **Hot Cluster:** Active, serving traffic.
- **Standby Cluster:** Ready to take over if hot fails.
- **Traffic Management:** Global load balancer routes traffic to the hot by default, standby if failure.
- **Data & Config Sync:** Databases, object storage, and deployments replicated between clusters.

---

2. Architecture Components

A. Kubernetes Clusters

- **Hot Cluster**
    - Multi-AZ for resilience.
    - Hosts all production workloads.
    - Stateful workloads use **replicated persistent volumes**.
- **Standby Cluster**
    - Mirrors hot cluster.
    - Sync deployments via Github Action CI + Fluxcd .

    - Can be hot-standby (near-live) or cold-standby (manual start).

B. Load Balancers

- **Global Load Balancer / DNS-based (Azure front door)**
    - Sends traffic to the hot cluster by default.
    - Fails over to the standby cluster if the hot fails.

- **Cluster-level Load Balancer**

    Fronts the Kubernetes cluster (opensource NGINX Ingress controller).

    - Routes internal traffic to services.

C. Network & Security

- Separate VPC/subnets per cluster.
- VPC peering or VPN between clusters.
- Security measures:
    - Kubernetes **NetworkPolicies** (pod-level access control)
    - TLS encryption for all traffic
    - RBAC for Kubernetes access
    - WAF (Azure front door)

D. Data Layer

- **Databases:** Multi-master or replicated (Postgres streaming, MySQL group replication)
- **Object storage:** Azure Blob replication
- **Sync:** CI/CD pipelines keep deployments in sync

E. Monitoring & Health Checks

- **One extra separate** cluster for **Prometheus + Grafana ,** monitor the hot and cold cluster
- Alertmanager in Prometheus + Azure Monitor

---

3. Hot-Standby Traffic Flow

1. User → Global Load Balancer
2. Default: Hot Cluster → Services → Databases / Storage
3. If Hot Cluster fails → LB reroutes → Standby Cluster
4. Standby Cluster already has synced apps + replicated data → minimal downtime

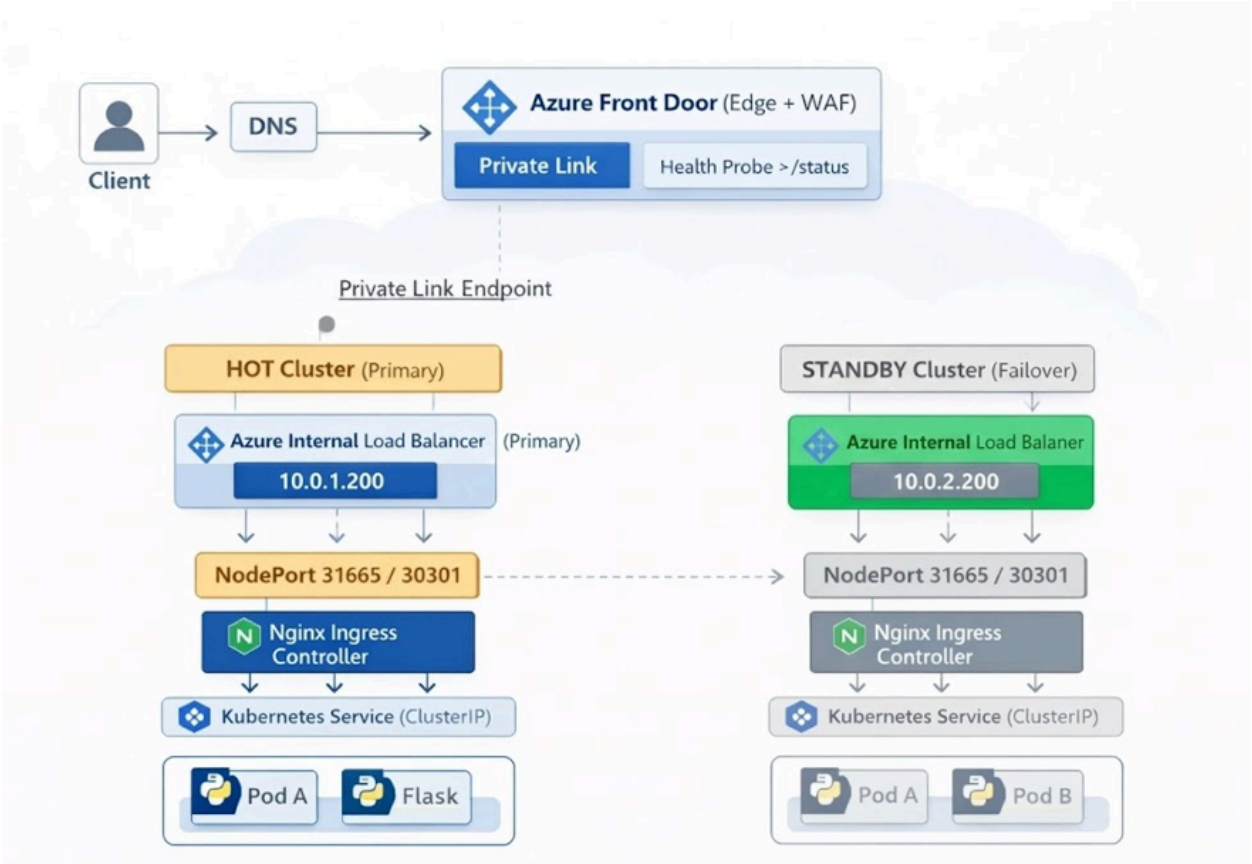
---

4. Best Practices

- **Kubernetes**
  - Multi-AZ clusters
  - Deployments for stateless apps, StatefulSets for DBs
  - Fluxcd HA (bootstrap on both hot and standby cluster, same git repo , different directory **/hot , **/standby)
- **Load Balancing**
  - Health checks at multiple layers
  - DNS failover for global HA(Azure front door health probe)
- **Data Management**

    - Async replication for heavy writes
    - Regular failover testing
- **Security**
    - TLS everywhere
    - Least-privilege RBAC
    - Audit logging
- **Monitoring**
    - Centralized logs (Loki Promtail + Grafana)
    - Alerts for failover events

Overall Architecture



K8s Cluster

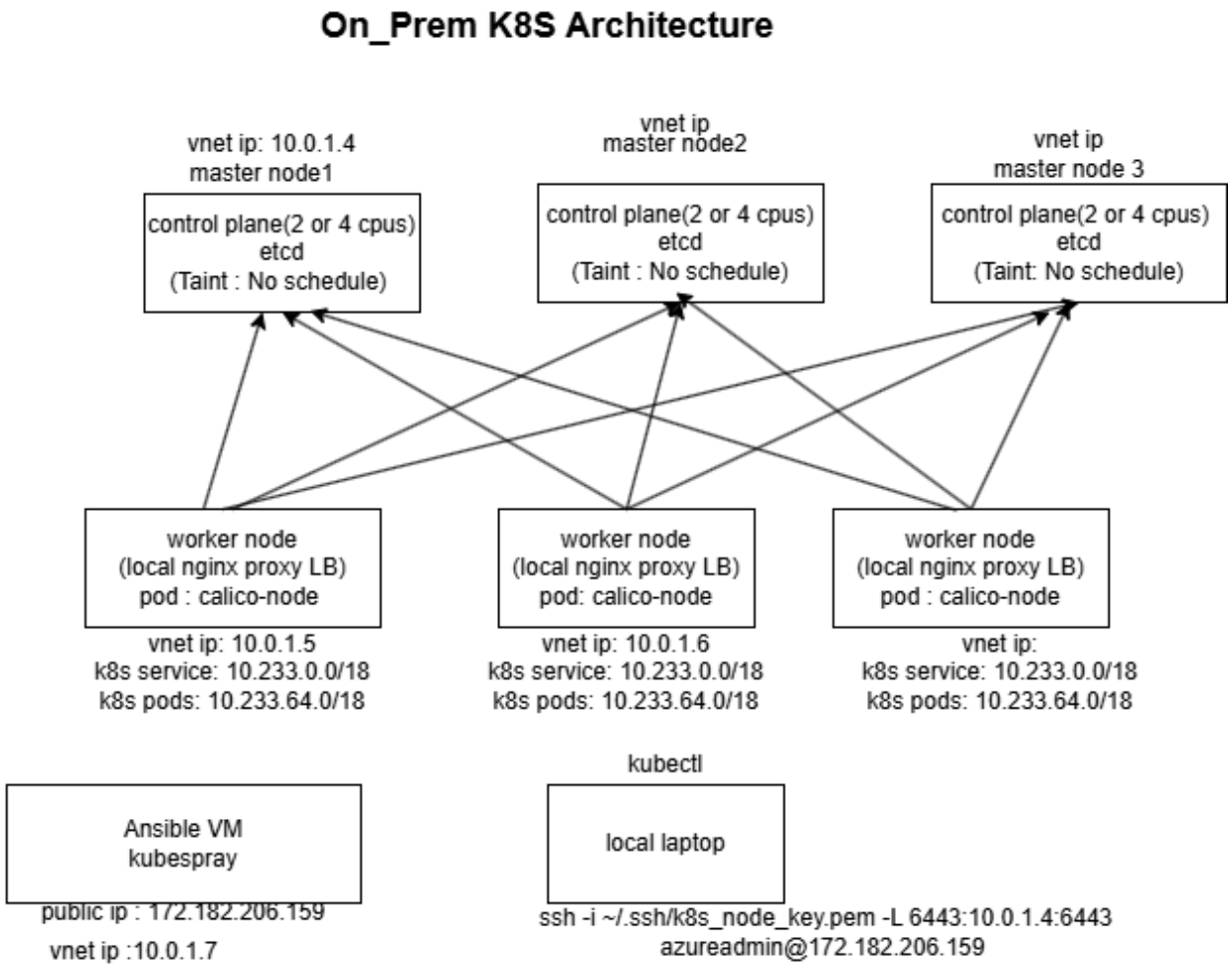
In this assignment, **self setup k8s** used instead of cloud native k8s(like azure aks).

nodes : azure vm , generated by **terraform**
vnet: "10.0.0.0/16"
subnet: "10.0.1.0/24"
nsg:

Inbound Security Rules

100	AllowInternal	Any	Any	VirtualNetwork	VirtualNetwork	Allow
200	AllowSSHFromAdmin	22	Tcp	204.244.241.200	Any	Allow
210	AllowK8sAPI	6443	Tcp	204.244.241.200	Any	Allow

Ansible Automation based on opensource kubespray.



For my lab , not all nodes are deployed as shown in the picture to save costs.

Install
ansible-playbook -i inventory/mycluster/inventory.ini -u azureadmin -b cluster.yml

```
(.vnev) (base) bei@LAPTOP-LJ5670Q7:/mnt/c/Users/beibe/Downloads/BMO/backend-app$ kubectl get nodes -owide
NAME    STATUS   ROLES           AGE     VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION      CONTAINER-RUN
TIME
node1   Ready    control-plane   4d19h   v1.35.1   10.0.1.4      <none>        Ubuntu 22.04.5 LTS   6.8.0-1044-azure    containerd://
2.2.1
node2   Ready    <none>          4d19h   v1.35.1   10.0.1.6      <none>        Ubuntu 22.04.5 LTS   6.8.0-1044-azure    containerd://
2.2.1
node3   Ready    <none>          4d19h   v1.35.1   10.0.1.5      <none>        Ubuntu 22.04.5 LTS   6.8.0-1044-azure    containerd://
2.2.1
node4   Ready    <none>          5h10m   v1.35.1   10.0.1.8      <none>        Ubuntu 24.04.3 LTS   6.14.0-1017-azure   containerd://
```

Network config (CNI plugin , k8s service/pods cidr etc)
inventory/mycluster/group_vars/k8s_cluster/k8s-cluster.yml
kube_network_plugin: calico
kube_service_addresses: 10.233.0.0/18
kube_pods_subnet: 10.233.64.0/18

metalLB config

```
(base) bei@LAPTOP-LJ5670Q7:/mnt/c/Users/beibe/Downloads/BMO/kubespray/inventory/mycluster/group_vars/metallb$ cat metallb-config.yaml

apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: vpc-ip-pool
  namespace: metallb-system
spec:
  addresses:
  - 10.0.1.90-10.0.1.100
---
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: l2-advert
  namespace: metallb-system
spec:
  ipAddressPools:
  - vpc-ip-pool
```

Nginx ingress controller

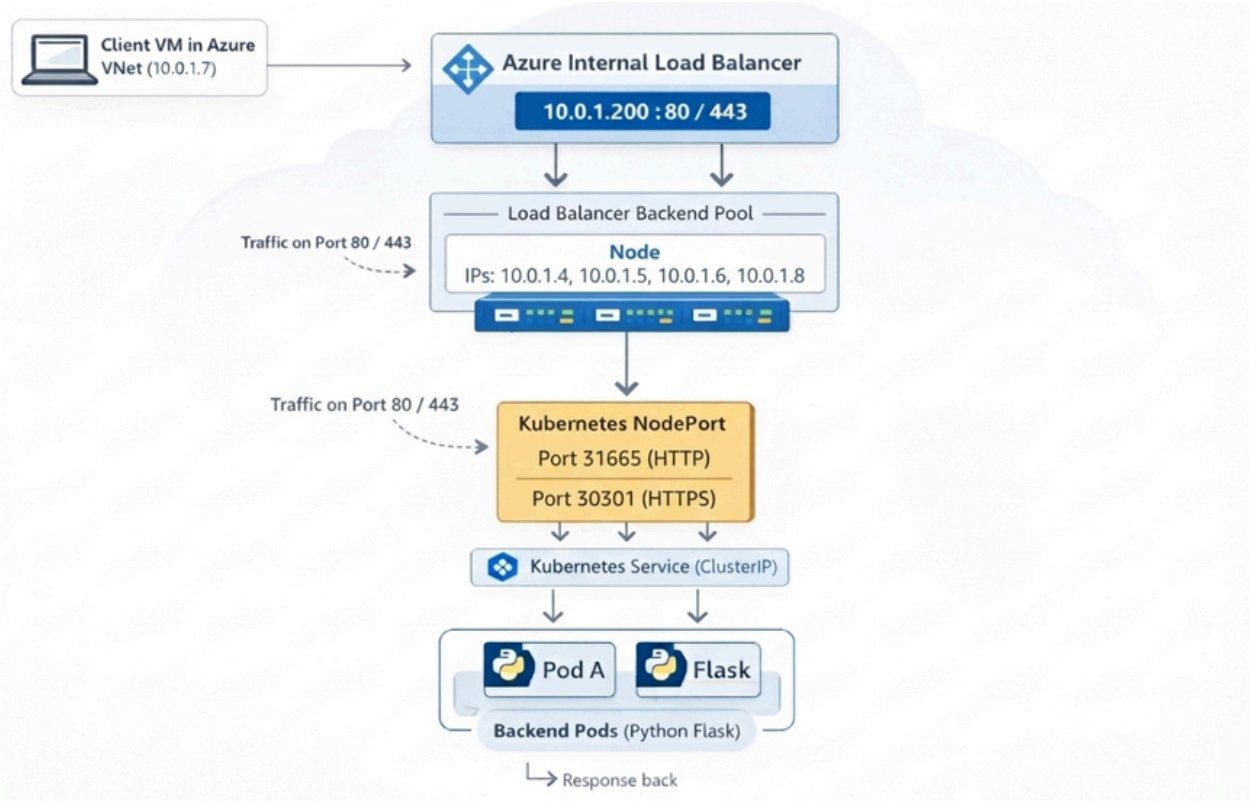
```
(base) bei@LAPTOP-LJ5670Q7:/mnt/c/Users/beibe/Downloads/BMO/kubespray/inventory/mycluster/group_vars/metallb$ kubectl get svc -n ngin
x-ingress-controller
NAME                                TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)                      AGE
nginx-ingress-controller-controller   LoadBalancer   10.233.12.107   10.0.1.90     80:31665/TCP,443:30301/TCP   31m
(base) bei@LAPTOP-LJ5670Q7:/mnt/c/Users/beibe/Downloads/BMO/kubespray/inventory/mycluster/group_vars/metallb$ kubectl get kustomizati
ons -A
NAMESPACE     NAME          AGE   READY   STATUS
flux-system   backend-app   32h   True    Applied revision: fflux@sha1:2f1b092e2aeb67f9dcb26f670a880015d886debd
flux-system   flux-system   34h   True    Applied revision: fflux@sha1:2f1b092e2aeb67f9dcb26f670a880015d886debd
(base) bei@LAPTOP-LJ5670Q7:/mnt/c/Users/beibe/Downloads/BMO/kubespray/inventory/mycluster/group_vars/metallb$ kubectl get ingress -n
bmo
NAME              CLASS    HOSTS           ADDRESS     PORTS   AGE
backend-ingress   <none>   backend.local   10.0.1.90   80      4m38s
```

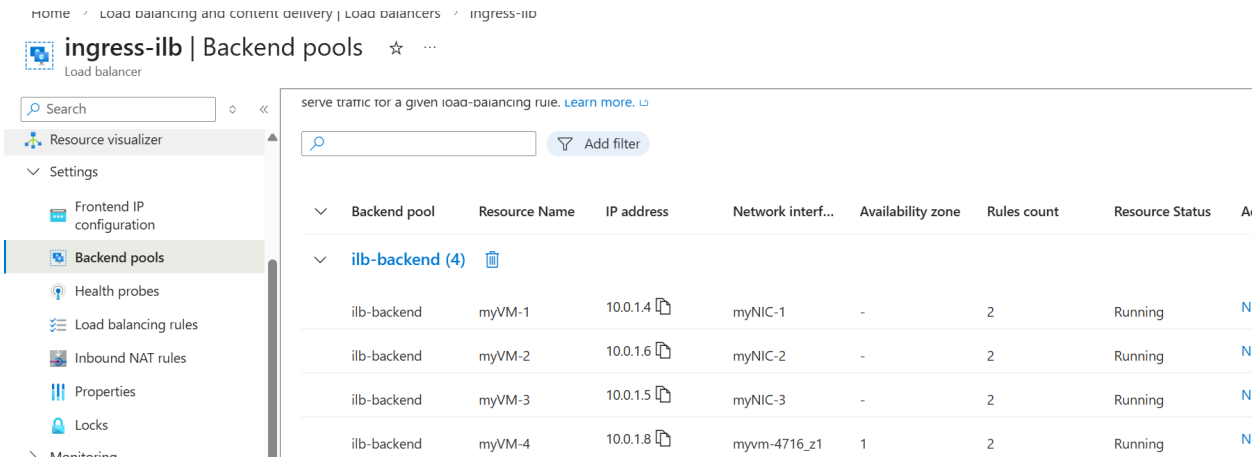
```
azureadmin@hostname-0:~/.ssh$ sudo vim /etc/hosts
azureadmin@hostname-0:~/.ssh$ cat /etc/hosts
10.0.1.90 backend.local
127.0.0.1 localhost

# The following lines are desirable for IPv6 capable hosts
::1 ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
ff02::3 ip6-allhosts
azureadmin@hostname-0:~/.ssh$ curl http://backend.local/status
curl -X POST http://backend.local/compute -H "Content-Type: application/json" -d '{"value":5}'
curl -X POST http://backend.local/stress -H "Content-Type: application/json" -d '{"duration":5,"threads":2}
curl: (7) Failed to connect to backend.local port 80 after 3084 ms: No route to host
curl: (7) Failed to connect to backend.local port 80 after 3068 ms: No route to host
curl: (7) Failed to connect to backend.local port 80 after 3068 ms: No route to host
```

MetalLb not working , root cause : MetalLB Layer2 mode relies on ARP advertisement to announce the service IP.
This works in bare-metal environments where all nodes share the same Layer-2 broadcast domain.
Azure Virtual Networks do not provide a true Layer-2 network. Traffic flows through Azure's software-defined routing layer:VM → Azure virtual router → VM

Azure internal Load Balancer
Change the type of nginx ingress controller to NodePort





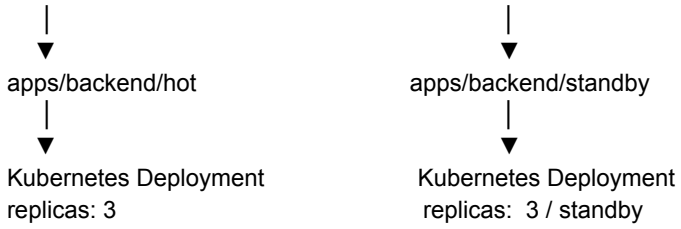

```
azureadmin@hostname-0:~$ curl -X POST http://backend.local/compute -H "Content-Type: application/json" -d '{"value":5}'
{"result":500}
azureadmin@hostname-0:~$ curl http://backend.local/status
{"status":"backend running"}
azureadmin@hostname-0:~$ cat /etc/hosts
10.0.1.200 backend.local
```

Upgrade : upgrade-cluster.yml

CICD pipeline

```
CI Pipeline
│
│ build & push
▼
Azure Container Registry
│
│ (image tags)
▼
Flux Image Policy
│
│ update tag
▼
Git Repo
(ha-gitops-repo)
│
├──────────────────────────────────────┤
│                                      │
▼                                      ▼
HOT Cluster                            STANDBY Cluster
(active traffic)                       (passive replica)

Flux Controllers                       Flux Controllers
├─ source-controller                   ├─ source-controller
├─ kustomize-controller                ├─ kustomize-controller
└─ image-automation-controller         └─ image-automation-controller
```



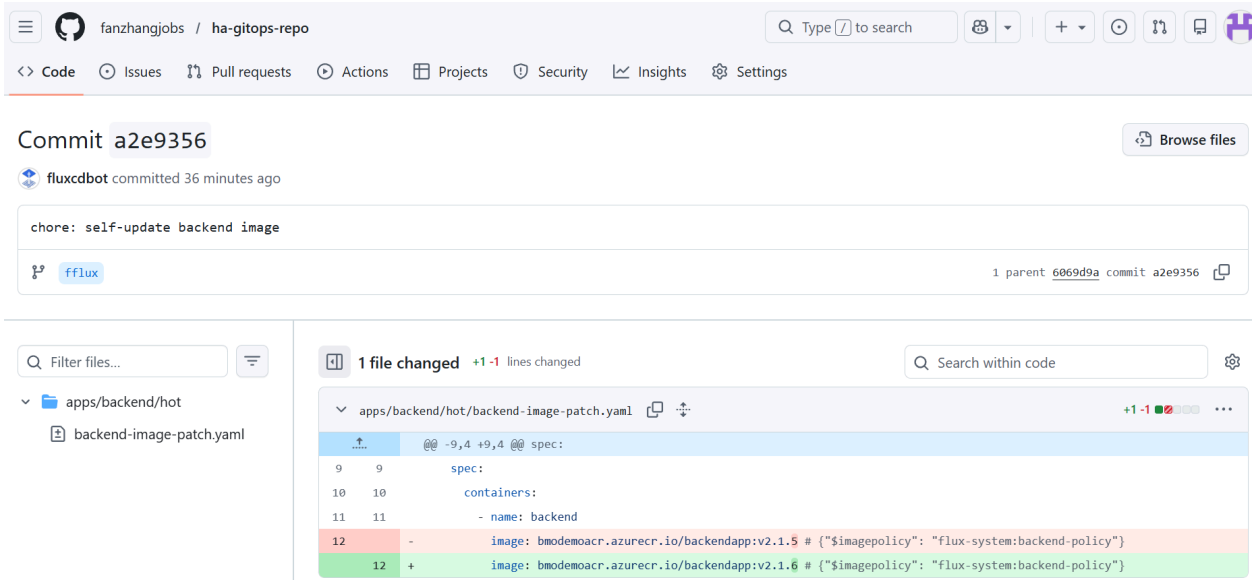

```
(base) bei@LAPTOP-LJ567OQ7:/mnt/c/Users/beibe/Downloads/BMO/ha-gitops-repo$ tree
.
├── apps
│   └── backend
│       ├── base
│       │   ├── acr-secret.yaml
│       │   ├── backend-deployment.yml
│       │   └── kustomization.yaml
│       ├── hot
│       │   ├── backend-image-patch.yaml
│       │   └── kustomization.yaml
│       └── standby
│           └── kustomization.yaml
└── clusters
    └── hot
        └── flux-system
            ├── app-sync.yaml
            ├── gitrepository.yaml
            ├── gotk-components.yaml
            ├── gotk-sync.yaml
            ├── image-automation.yaml
            ├── image-policy.yaml
            └── kustomization.yaml
```

Github two repos ::
application: https://github.com/fanzhangjobs/backend-app (Dockerfile included)
Fluxcd: https://github.com/fanzhangjobs/ha-gitops-repo (k8s manifest included)

reusable base manifest ,Hot Environment Overlay (patch), Standby Environment Overlay (patch)

CI
application repo .github/workflows/build-and-push.yml
Run pytest ->build ->push to azure ACR ,image tag (support auto increment like 2.1.1 to 2.1.2 if no manual tag by github environment variable)

CD
Fluxcd (image update automation once new image pushed)



Note: For secrets.yaml we should encrypt it. Sops tool used . decryption needed in Kustomization in Fluxcd

Monitoring and logging

- Health probe in Azure Front Door (required)
- Azure front door , load balancer , configure Monitoring ->Diagnostic settings->Send to Log Analytics workspace (KQL to query , filter logs)
- K8s cluster : Prometheus + grafana

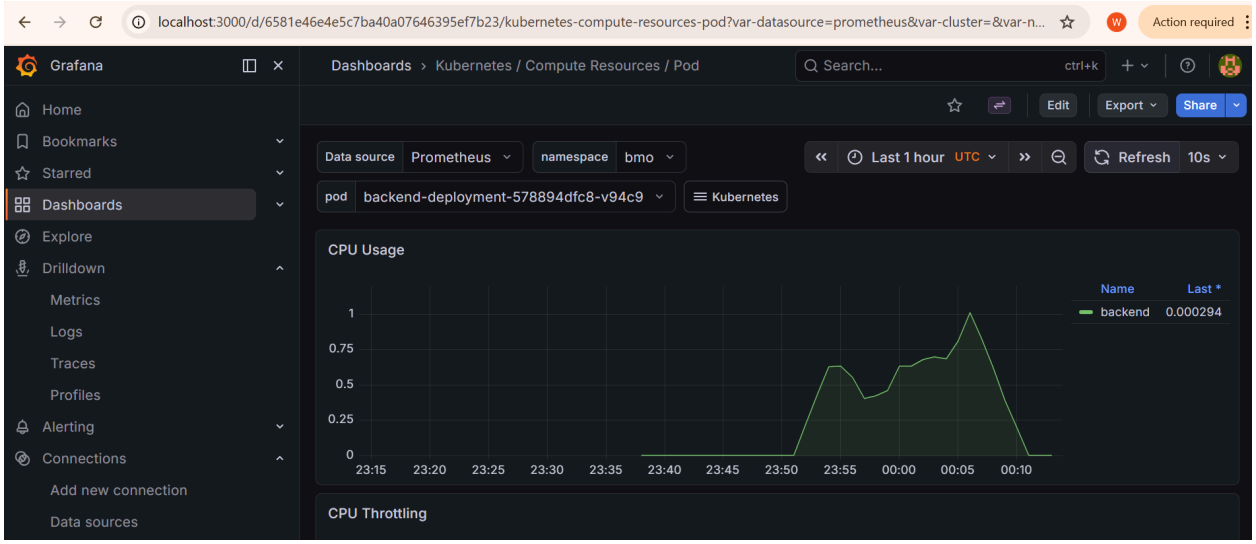
In my lab I didn't create an extra cluster to save cost. In production, extra clusters are needed.

Install helm chart prometheus-community/kube-prometheus-stack
grafana data source , connection , Prometheus server URL ,
http://prometheus-kube-prometheus-prometheus.monitoring.svc.cluster.local:9090

manually triggered cpu stress to monitor the cpu usage

```
(base) bei@LAPTOP-LJ5670Q7:/mnt/c/Users/beibe/Downloads/BMO/backend-app$ kubectl port-forward svc/backend-service -n bmo 4000:80
Forwarding from 127.0.0.1:4000 -> 5000
Forwarding from [::1]:4000 -> 5000
```

```
(.vnev) (base) bei@LAPTOP-LJ5670Q7:/mnt/c/Users/beibe/Downloads/BMO/prometheus$ curl -X POST http://localhost:4000/stress   -H "Content-Type: ap
plication/json"   -d '{"duration":300, "threads":20}'
{"status":"CPU stress started for 300s with 20 threads"}
```



- Shared central logging, Promtail → Loki → Grafana, Both clusters send logs to one Loki.

Todo:

- Set a standby cluster , configure azure front door to test the failover
- More complicated application(including db …)
- Test on k8s ha(master node failover)
- Network policy on k8s(now i only have one backend app in test)
- Dns name, certificate , https